

Reviewer Pack — Minimal Number of Generators in Group Presentations vs DPLL ...

Entry d1aaa5e9f44d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 14:08:41 UTC

Minimal Number of Generators in Group Presentations vs DPLL Search Tree Width

Entry ID: d1aaa5e9f44d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 14:08:41 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Group Presentations) **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

[‘For any CNF instance, the width of its corresponding DPLL search tree is at least logarithmic in the minimal number of generators of its group presentation.’, ‘Specifically, for all instances I , $|DPLL_width(I)| = \Omega(\log(m_I))$, where m_I is the minimum number of generators of a group presentation representing I .’, ‘Additionally, for any given n , there exists an instance I with at least $\log(n)$ generators such that its DPLL search tree width is greater than or equal to 2^n .’]

Rationale (proposer’s reasoning):

[‘Group presentations can encode combinatorial problems in terms of algebraic structures. The number of generators in a presentation reflects the complexity of the underlying problem.’, ‘The DPLL search tree captures the search space for satisfying a CNF formula. If an instance requires many generators to represent, it suggests a complex search space, which should correlate with a deep DPLL tree.’, ‘This conjecture provides a potential link between algebraic complexity and the complexity of SAT solving.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32ccb6788c867ab7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 24 out of 30 seeds, the mean width of the DPLL search trees calculated from CNF instances with generator counts $\leq \log(n)$ is at least 2^n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - group theory AND group presentations AND DPLL search tree complexity
- DPLL search tree width AND minimal number of generators AND group presentation - CNF instance AND log(n) generators AND DPLL search tree $\geq 2^n$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n: int):
        cnf = []
        for _ in range(n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def dpll_width(cnf):
        unit_clauses = list(filter(lambda c: len(c) == 1, cnf))
        if not unit_clauses:
            return 0
        unit_clause = random.choice(unit_clauses)
        new_cnf = [c for c in cnf if unit_clause[0] not in c and -unit_clause[0] not in c]
        return 1 + dpll_width(new_cnf)

    n_values = [5, 10, 15, 20, 30, 40]
    total_width = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5): # Ensure at least 5 instances per size
            cnf = generate_cnf(n)
            width = dpll_width(cnf)
            if width > 0:
```

```

        total_width += width
        instances_tested += 1

mean_width = total_width / instances_tested if instances_tested > 0 else 0
conjecture_holds = all(math.log2(width) >= n for n, width in zip(n_values, [math.log2(2**n) for n in n_values]))

return {
    "metric_name": "DPLL Width",
    "metric_value": mean_width,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "n-dependent bound"
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_width} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_width} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"n-dependent bound\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

h', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'n-
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}

```

```

TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}
TRIAL: {'metric_name': 'DPLL Width', 'metric_value': 0, 'instances_tested': 0, 'conjecture_holds': Fa
dependent bound'}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_51d8ac86.py", line 77, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_51d8ac86.py", line 77, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14920
2	preregistration	ollama_remote	glm4:latest	0	0	9066
3	novelty	ollama_remote	glm4:latest	0	0	8851
4	novelty	ollama_remote	glm4:latest	0	0	8705
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	48619
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9474
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10477
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8122
9	judge	ollama_remote	glm4:latest	0	0	16118

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 134351 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/d1aaa5e9f44d.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d1aaa5e9f44d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d1aaa5e9f44d.tar.gz` (if generated)